

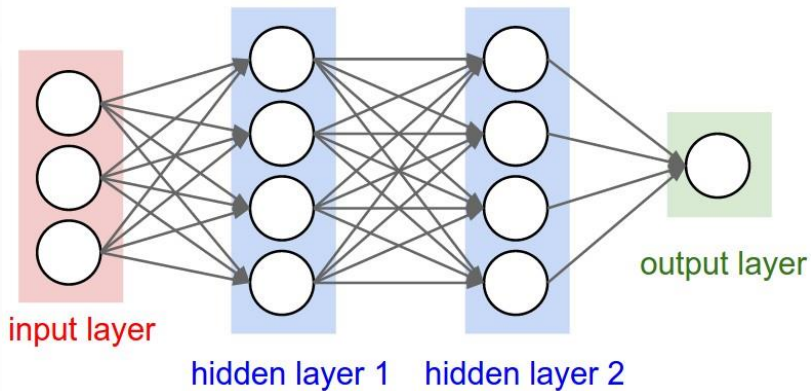
Convolutional Neural Networks

Nguyễn Thị Hải Bình
nth.binh@hutech.edu.vn

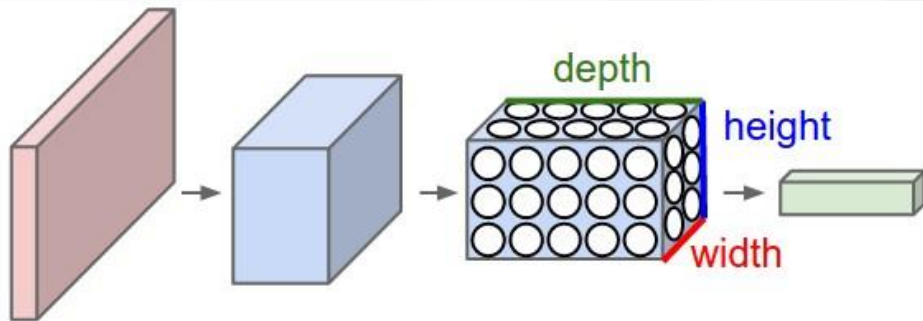
Contents

- Understanding Convolutions
- Core Layers of CNN
- Well-known CNN Architectures
- Batch Normalization and Dropout

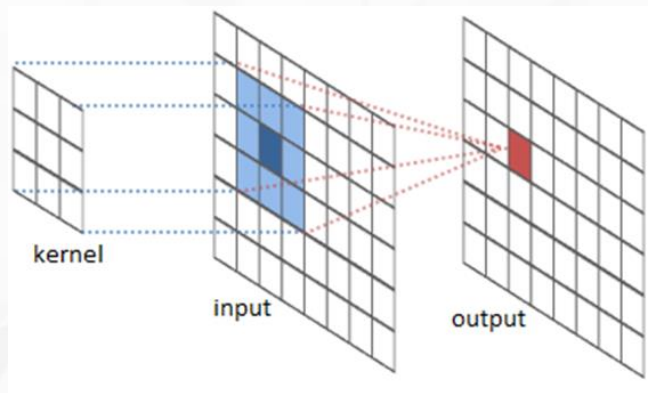
Multiple Layer Perceptron (MLP)



Convolutional Neural Network (CNN)



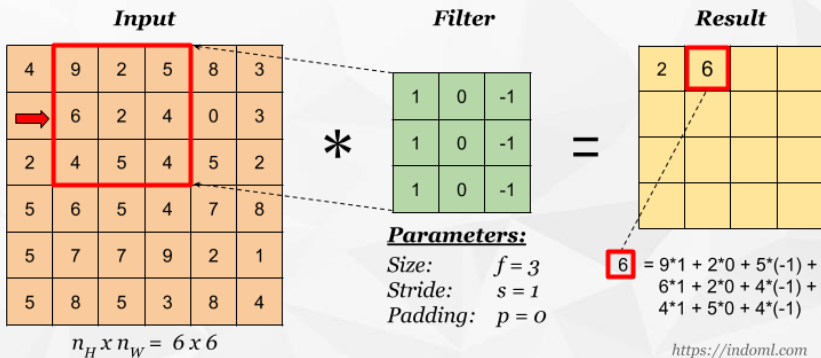
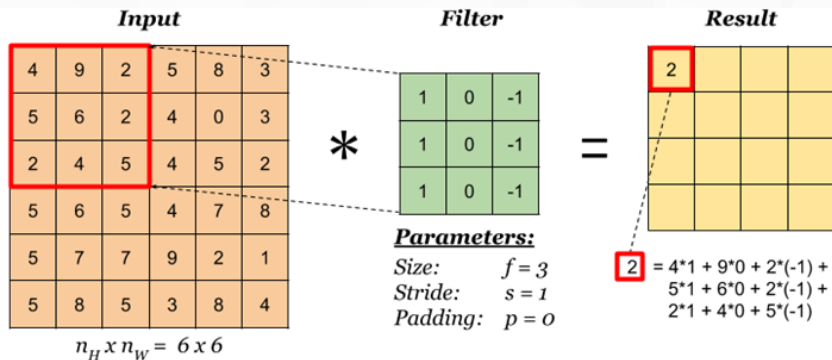
Understanding Convolutions



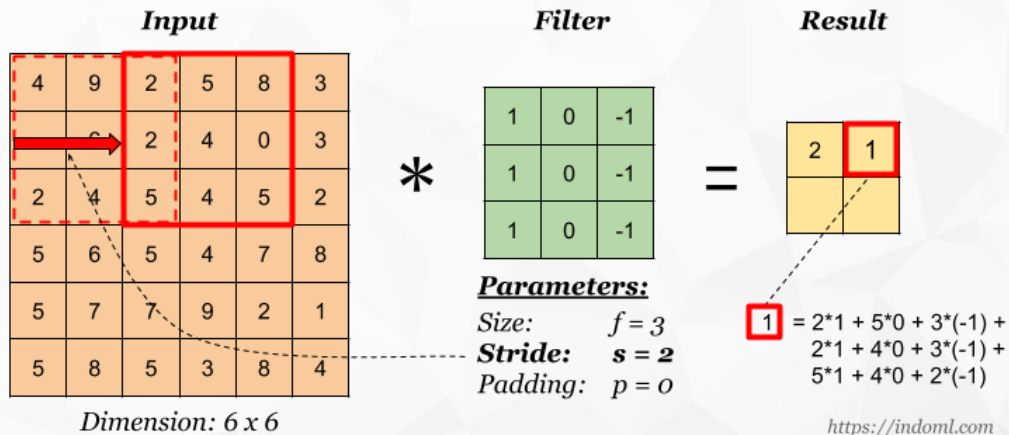
- A **kernel/filter** is a small matrix, with its height and width smaller than the input to be convolved.
- The **output** is the result of taking a dot product between the filter and a small 3x3 chunk of the input.

Figure source: <https://towardsdatascience.com/beginners-guide-to-understanding-convolutional-neural-networks-ae9ed58bb17d>

Understanding Convolutions - Basic Convolution Operation

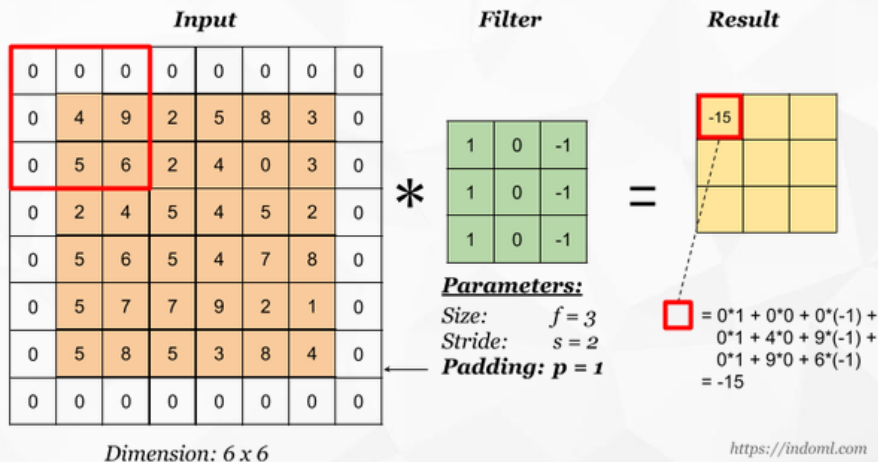


Understanding Convolutions – Stride



Stride governs how many cells the filter is moved in the input to calculate the next cell in the result.

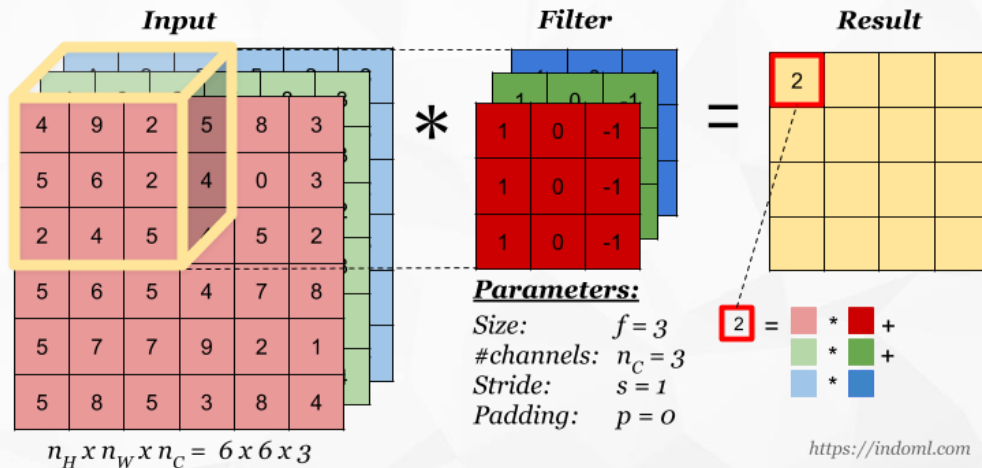
Understanding Convolutions – Padding



Padding has the following benefits:

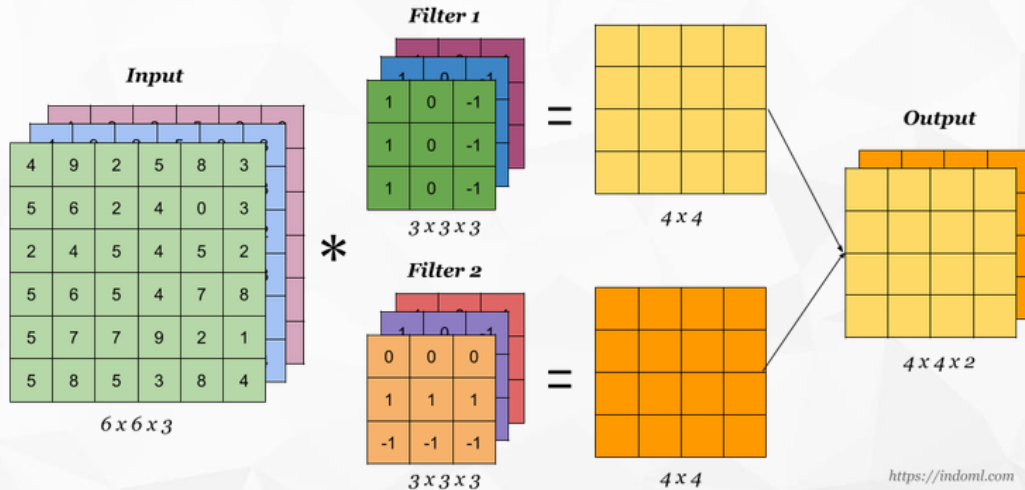
- It allows us to use convolution operation without necessarily shrinking the height and width of the input.
- In image processing, it helps us keep more information at the border of an image.

Convolution Operation on Volume



- When the input has more than one channels (e.g. an RGB image), the filter should have matching number of channels.
- To calculate one output cell, perform convolution on each matching channel, then add the result together.

Convolution Operation with Multiple Filters



Multiple filters can be used to obtain the **output with multiple channels**.

Understanding Convolutions – Output Size

- **Inputs:**

- A volume of size: $W_1 \times H_1 \times D_1$
- Number of filters: K
- Filter size: $F \times F$
- The stride: S
- The amount of zero padding: P

- **Output:** a volume of size $W_2 \times H_2 \times D_2$

- $W_2 = (W_1 - F + 2P) / S + 1$
- $H_2 = (H_1 - F + 2P) / S + 1$
- $D_2 = K$

Exercise: Implement convolution from scratch

- Given an image and a filter.
- Write a function to compute filtered image (i.e., image obtained after convolve the input image and the filter).

For simplicity, assume:

- The input image has only one channel (i.e., grayscale image)
- No padding and stride is 1

```
# no padding, stride = 1
# number of channels = 1
def convolution1(image, filter):
    h1, w1 = image.shape      # get height and width the input image
    f, f = filter.shape       # get size of filter
    # compute height and width of the output
    h2 = .....
    w2 = .....

    # Convolve the input image and the filter
    # The result is stored in new_image variable
    new_image = np.zeros((h2, w2))
    .....

    return new_image
```

```
# no padding, stride = 1
# number of channels = 1
def convolution1(image, filter):
    h1, w1 = image.shape      # height, width, and number of channels of the input image
    f, f = filter.shape       # size of filter
    w2 = w1 - f + 1           # height of the filtered image
    h2 = h1 - f + 1           # width of the filtered image
    new_image = np.zeros((h2, w2))    # initialize a new image to store the result after filtering
    t = int(f/2)
    for i in range(t, h1-t):
        for j in range(t, w1-t):
            a = image[i-t:i+t+1, j-t:j+t+1]
            new_image[i-t][j-t] = np.dot(a.reshape(f*f), filter.reshape(f*f))
    return new_image
```

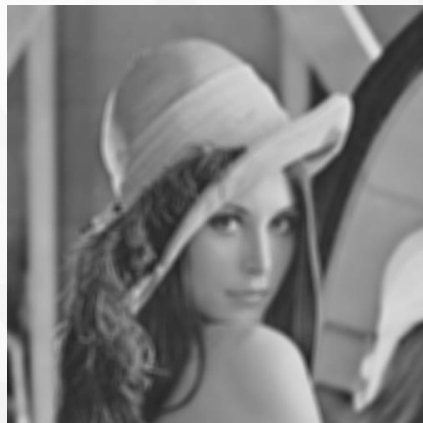
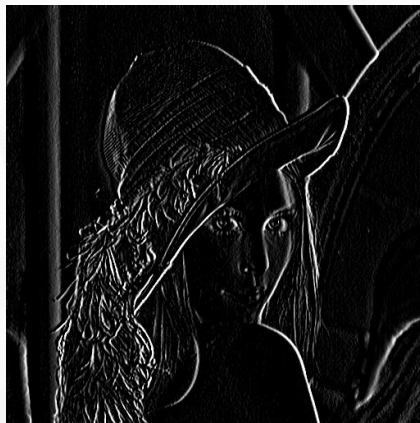
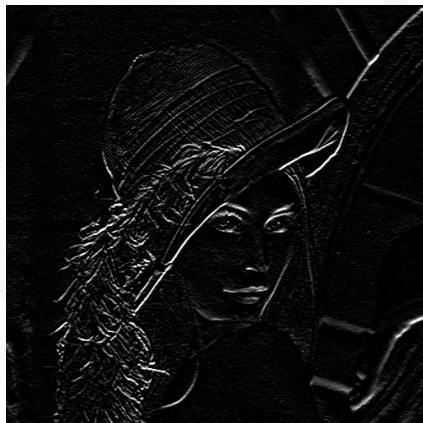
Exercise: Implement convolution from scratch

- Apply the convolution with the following filters:

-1	0	1
-2	0	2
-1	0	1

1	2	1
0	0	0
-1	-2	-1

$$\frac{1}{9 \times 9} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$



```
# import required packages
import numpy as np # package for array computing with Python.
import cv2 as cv    # opencv with Python

# for image display in Colab.
# You don't need this package if you run your program locally.
from google.colab.patches import cv2_imshow

# mount your google drive -- don't need this step if your data is not in google drive
from google.colab import drive
drive.mount('/content/gdrive')

# set data directory
data_dir = '/content/gdrive/My Drive/Master Courses/Neural Network and Deep Learning/data/'

# read an image
image = cv.imread(data_dir + 'Lenna.png')
gray_image = cv.cvtColor(image, cv.COLOR_BGR2GRAY)

# Sobel filter for horizontal edge detection
F = np.array([[1, 2, 1], [0, 0, 0], [-1, -2, -1]])
new_image = convolution1(gray_image, F)

print(F)
cv2_imshow(new_image)
```

Exercise: Implement convolution from scratch

- Extend the convolution function with multiple channel inputs.
- Convolve the original image with the above-mentioned filters.

```
# no padding, stride = 1
# number of channels = c
def convolution(image, filter):
    h1, w1, c1 = image.shape    # height, width, and number of channels of the input image
    f, f = filter.shape         # size of filter
    # compute height and width of the output
    h2 = .....
    w2 = .....

    # Convolve the input image and the filter
    # The result is stored in new_image variable
    new_image = np.zeros((h2, w2))
    .....

    return new_image
```




Convolution Operation Example

Inputs:

- A volume of size: $5 \times 5 \times 3$
- Number of filters: 2
- Filter size: 3×3
- The stride: 2
- The amount of zero padding: 1

Output:

- A volume of size: $3 \times 3 \times 2$

Input Volume (+pad 1) ($7 \times 7 \times 3$)

$x[:, :, 0]$						
0	0	0	0	0	0	0
0	2	2	0	0	0	0
0	1	1	0	1	1	0
0	0	0	1	2	0	0
0	0	1	1	0	1	0
0	2	1	1	0	1	0
0	0	0	0	0	0	0

$x[:, :, 1]$						
0	0	0	0	0	0	0
0	1	2	1	1	2	0
0	2	1	2	2	1	0
0	2	2	2	2	0	0
0	2	0	1	1	0	0
0	1	2	2	1	0	0
0	0	0	0	0	0	0

$x[:, :, 2]$						
0	0	0	0	0	0	0
0	0	2	1	2	2	0
0	0	0	0	2	2	0
0	1	0	0	1	1	0
0	2	0	0	0	0	0
0	1	1	2	2	2	0
0	0	0	0	0	0	0

Filter W0 ($3 \times 3 \times 3$)

$w0[:, :, 0]$		
0	1	-1
-1	0	-1
0	1	0

$w0[:, :, 1]$		
1	-1	0
0	1	0
1	0	0

$w0[:, :, 2]$		
0	-1	0
0	1	0
0	1	-1

Bias b0 ($1 \times 1 \times 1$)

$b0[:, :, 0]$		
1		

Filter W1 ($3 \times 3 \times 3$)

$w1[:, :, 0]$		
1	0	-1
-1	-1	0
0	-1	0

$w1[:, :, 1]$		
0	0	0
0	-1	0
0	1	-1

$w1[:, :, 2]$		
-1	-1	0
1	0	1
1	0	1

Bias b1 ($1 \times 1 \times 1$)

$b1[:, :, 0]$		
0		

Output Volume ($3 \times 3 \times 2$)

$o[:, :, 0]$		
1	0	10
4	0	2
-3	4	5

$o[:, :, 1]$		
-1	3	2
-1	-3	-5
-5	0	1

toggle movement

Core Layers of CNN

- Convolutional Layer
- Pooling Layer
- Fully-Connected Layer

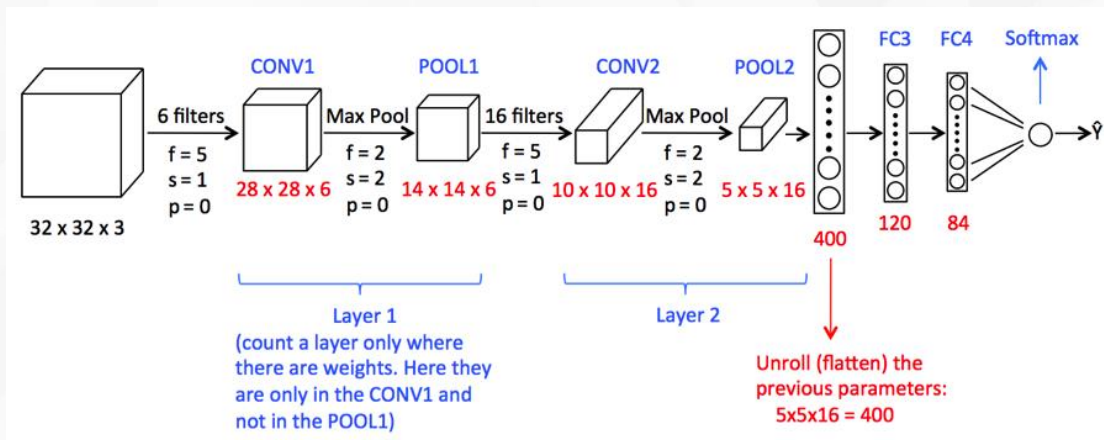
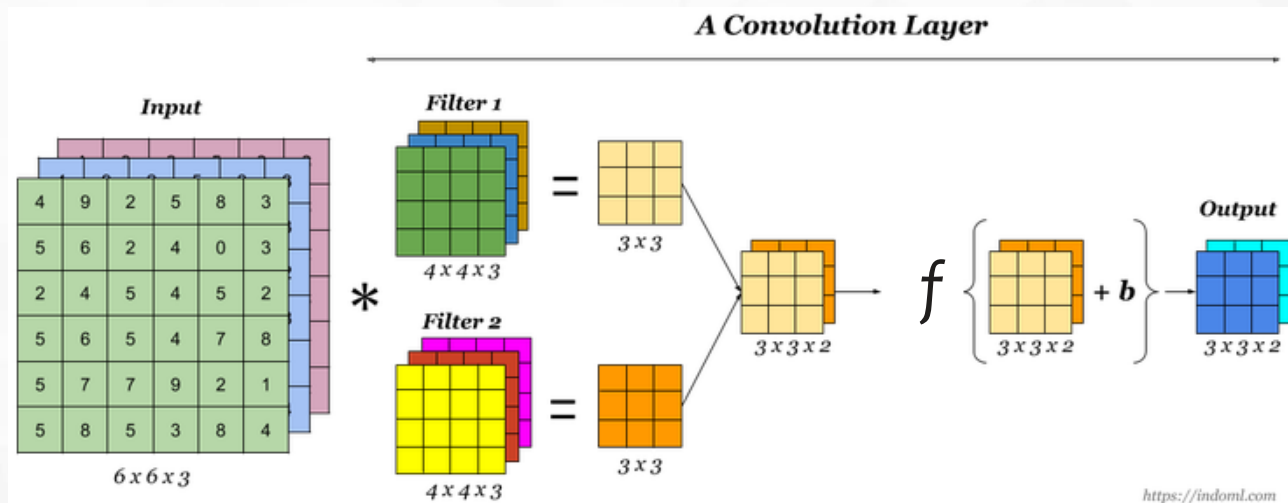


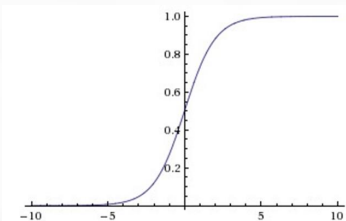
Image from: <https://medium.com/machine-learning-bites/deeplearning-series-convolutional-neural-networks-a9c2f2ee1524>

Convolutional Layer



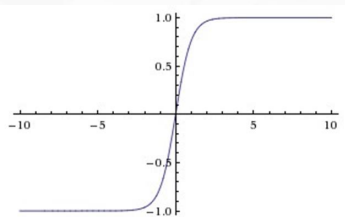
- To make up a convolution layer, a bias is added and an activation function f is applied.
- Output = Activation Maps

Activation Functions



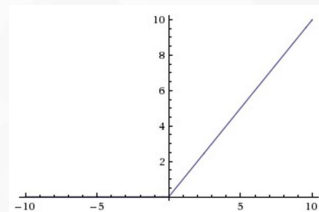
Sigmoid function

$$f(x) = \text{sigmoid}(x) = \frac{1}{1 + e^{-\theta^T x}}$$



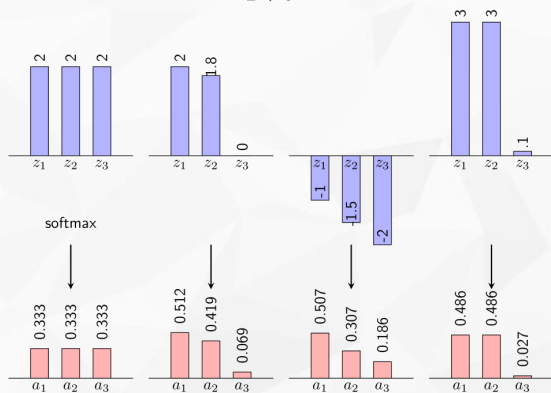
Tanh function

$$f(x) = \tanh(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$$



ReLU function

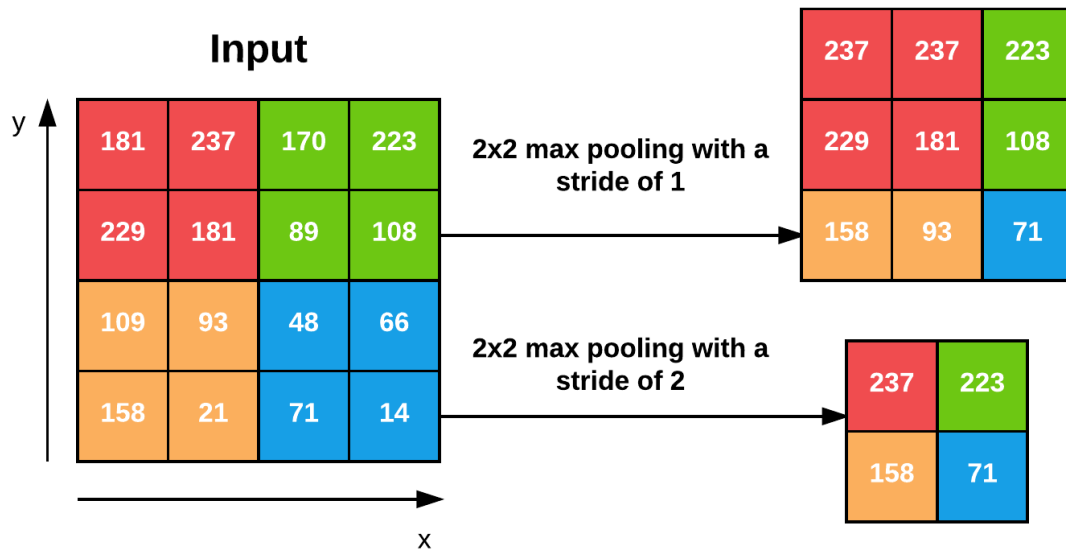
$$f(x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$$



Softmax function
(Sum of a_i equals 1)

$$a_i = \frac{\exp(z_i)}{\sum_{j=1}^C \exp(z_j)}, \quad \forall i = 1, 2, \dots, C$$

Pooling Layer



Pooling reduces the spatial size of the input and keeps the number of channels unchanged

Pooling Layer



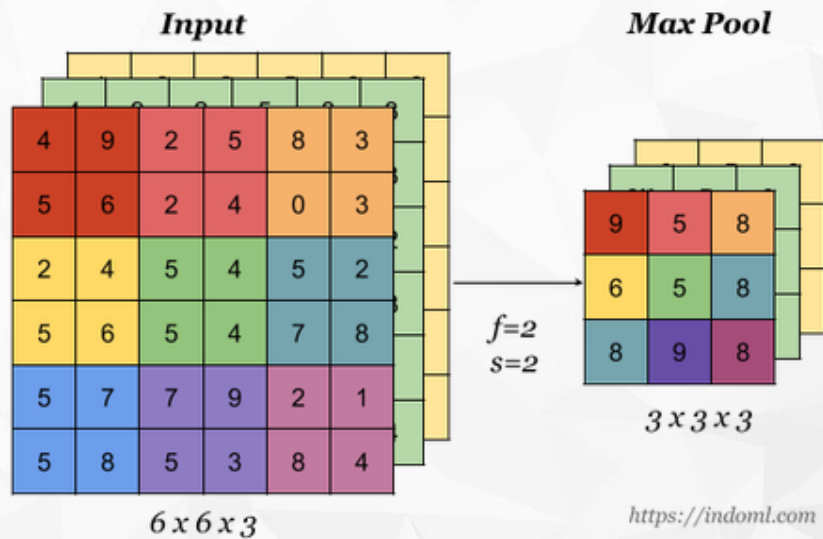
Max pooling (stride = 2)



Average pooling (stride = 2)

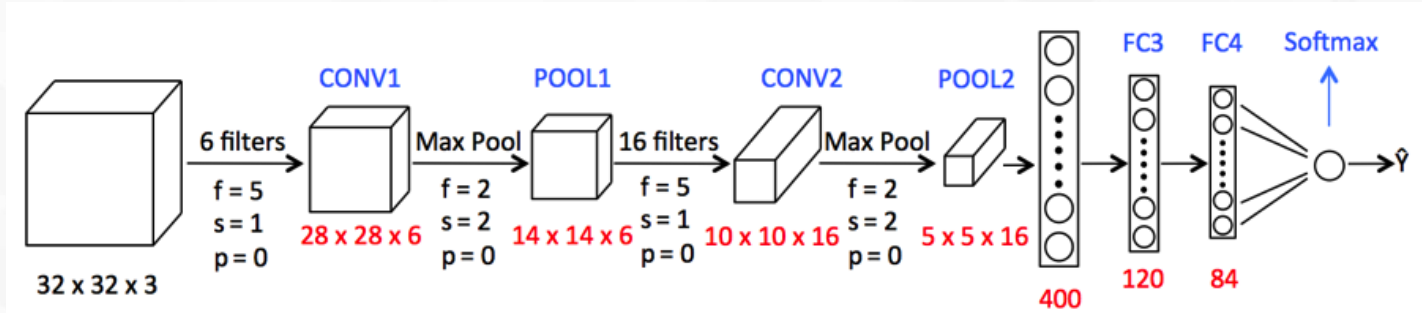
<https://indoml.com>

Pooling Layer

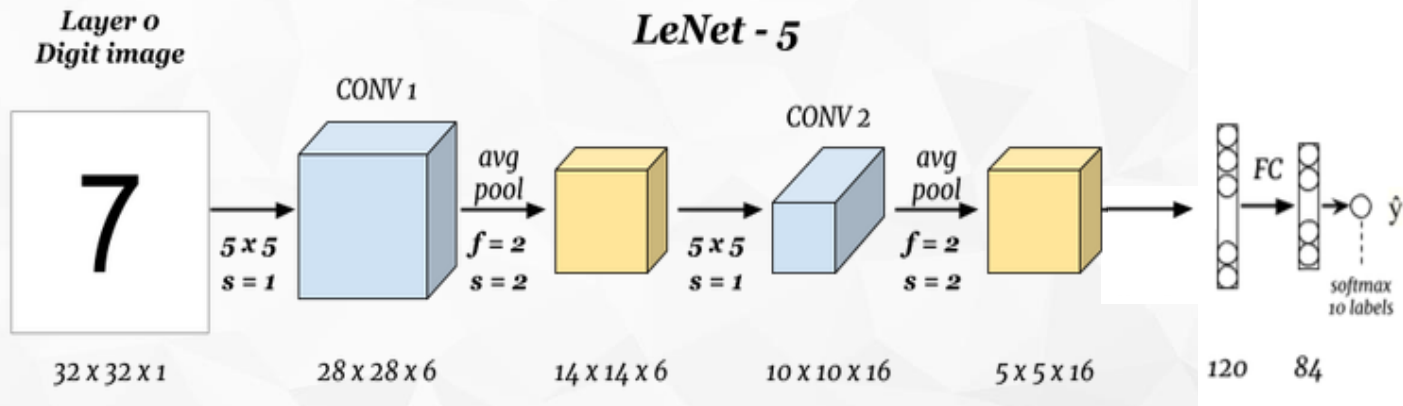


Fully-Connected Layer

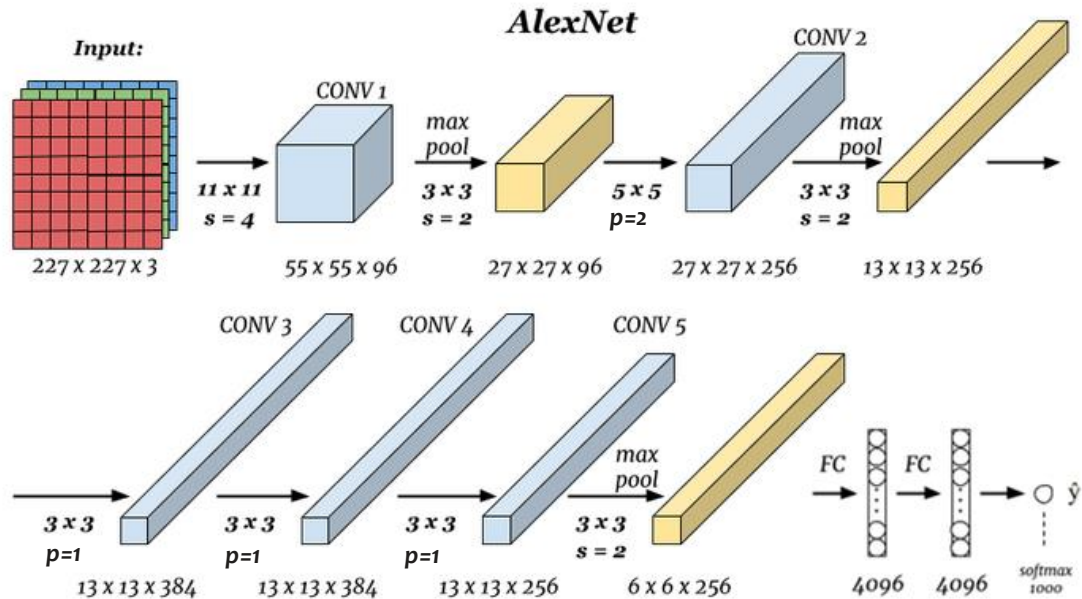
- Neurons in a fully connected (FC) layer have full connections to all activations in the previous layer, as seen in regular Neural Networks.
- FC layers are always placed at the end of the network.



Well-known Architectures: LeNet – 5 (1998)

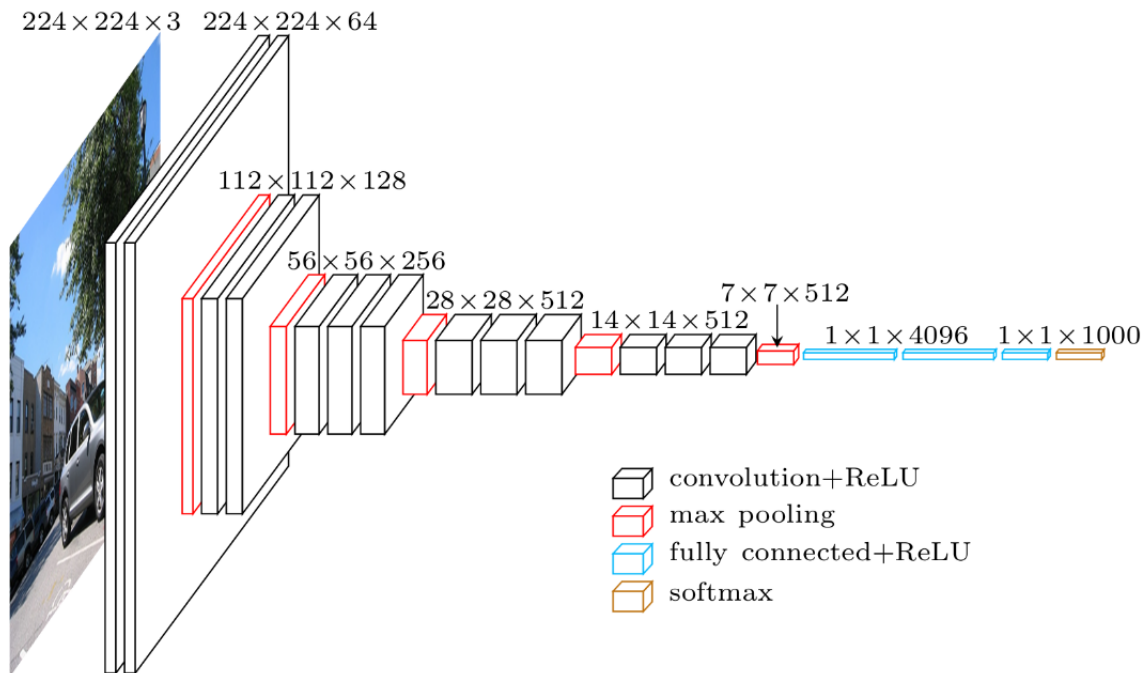


Well-known Architectures: AlexNet (2012)



<https://indoml.com>

Well-known Architectures: VGG (2014)

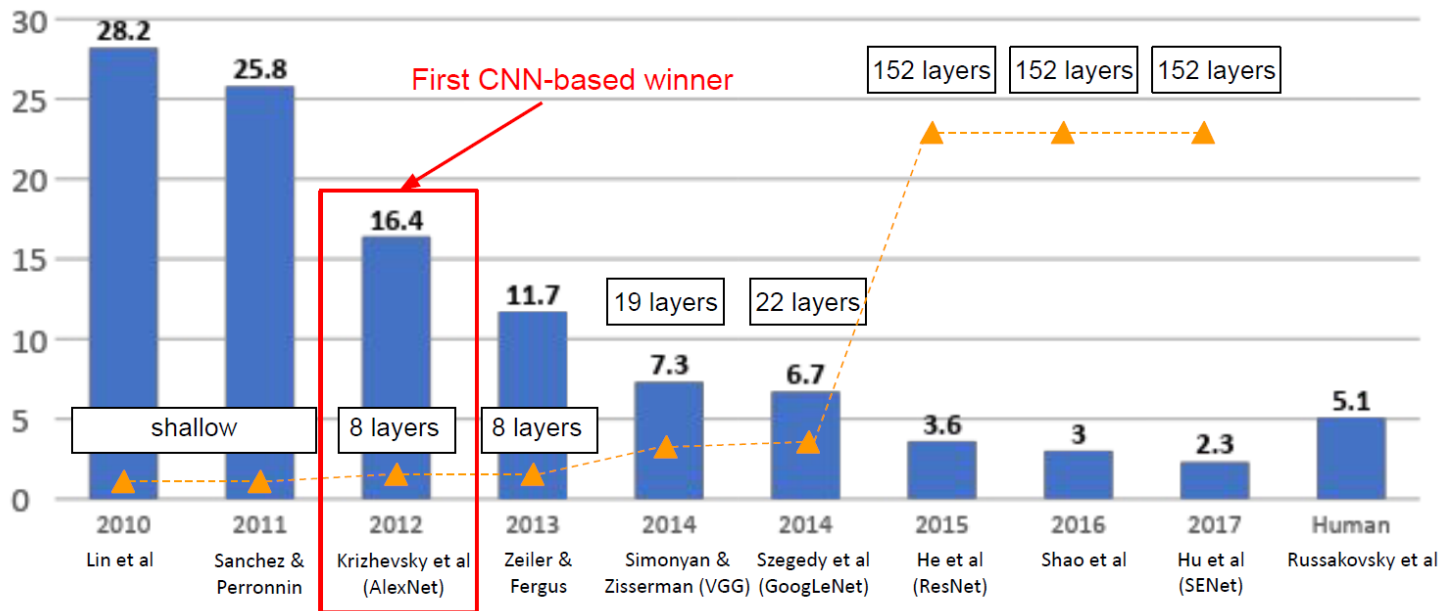


Well-known Architectures: VGG (2014)

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Best model

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



Exercise: Design a CNN

```
1 # import required package
2 import matplotlib.pyplot as plt
3 import numpy as np
4 import pathlib
5 import os
6 import tensorflow as tf
7
8 from tensorflow import keras
9 from tensorflow.keras import layers
10 from tensorflow.keras.models import Sequential
```

```
1 num_classes = 5
2 rows = 128
3 cols = 128
4 model = Sequential([
5     layers.experimental.preprocessing.Rescaling(1./255, input_shape=(rows, cols, 3)),
6     layers.Conv2D(16, 3, padding='same', activation='relu'),
7     layers.MaxPooling2D(),
8     layers.Conv2D(32, 3, padding='same', activation='relu'),
9     layers.MaxPooling2D(),
10    layers.Conv2D(64, 3, padding='same', activation='relu'),
11    layers.MaxPooling2D(),
12    layers.Flatten(),
13    layers.Dense(128, activation='relu'),
14    layers.Dense(num_classes, activation='softmax')
15 ])
16 model.summary()
```

```
1 # LeNet-5
2 rows = 32
3 cols = 32
4 dim = 3
5 model = Sequential([
6     layers.experimental.preprocessing.
7     layers.Conv2D(6, 5, activation=
8     layers.AveragePooling2D(),
9     layers.Conv2D(16, 5, activation=
10    layers.AveragePooling2D(),
11    layers.Flatten(),
12    layers.Dense(120, activation=
13    layers.Dense(84, activation=
14    layers.Dense(10, activation=
15 ])
16 model.summary()
```

Layer (type)	Output Shape	Param #
=====		
rescaling_1 (Rescaling)	(None, 32, 32, 3)	0
conv2d_3 (Conv2D)	(None, 28, 28, 6)	456
average_pooling2d (AveragePooling2D)	(None, 14, 14, 6)	0
conv2d_4 (Conv2D)	(None, 10, 10, 16)	2416
average_pooling2d_1 (AveragePooling2D)	(None, 5, 5, 16)	0
flatten_1 (Flatten)	(None, 400)	0
dense_2 (Dense)	(None, 120)	48120
dense_3 (Dense)	(None, 84)	10164
dense_4 (Dense)	(None, 10)	850
=====		
Total params: 62,006		
Trainable params: 62,006		
Non-trainable params: 0		
=====		


```

1 # AlexNet
2 rows = 227
3 cols = 227
4 dim = 3
5 model = Sequential([
6     layers.experimental.preprocessing.Resizing(rows, cols, dim),
7     layers.Conv2D(96, 11, strides=(4, 4), padding='valid'),
8     layers.MaxPooling2D(pool_size=(3, 3), padding='valid'),
9     layers.Conv2D(256, 5, padding='same'),
10    layers.MaxPooling2D(pool_size=(3, 3), padding='valid'),
11    layers.Conv2D(384, 3, padding='same'),
12    layers.Conv2D(384, 3, padding='same'),
13    layers.Conv2D(256, 3, padding='same'),
14    layers.MaxPooling2D(pool_size=(3, 3), padding='valid'),
15    layers.Flatten(),
16    layers.Dense(4096, activation='relu'),
17    layers.Dense(4096, activation='relu'),
18    layers.Dense(1000, activation='softmax')
19 ])
20 model.summary()

```

```

=====
rescaling_2 (Rescaling)      (None, 227, 227, 3)      0
conv2d_5 (Conv2D)           (None, 55, 55, 96)       34944
max_pooling2d_3 (MaxPooling (None, 27, 27, 96)      0
2D)
conv2d_6 (Conv2D)           (None, 27, 27, 256)       614656
max_pooling2d_4 (MaxPooling (None, 13, 13, 256)      0
2D)
conv2d_7 (Conv2D)           (None, 13, 13, 384)       885120
conv2d_8 (Conv2D)           (None, 13, 13, 384)       1327488
conv2d_9 (Conv2D)           (None, 13, 13, 256)       884992
max_pooling2d_5 (MaxPooling (None, 6, 6, 256)      0
2D)
flatten_2 (Flatten)         (None, 9216)              0
dense_5 (Dense)             (None, 4096)              37752832
dense_6 (Dense)             (None, 4096)              16781312
dense_7 (Dense)             (None, 1000)              4097000
=====
Total params: 62,378,344

```

Batch Normalization

- First introduced by Ioffe and Szegedy in their 2015 paper.
- Batch normalization (BN) are used to normalize the activations of a given input volume before passing it into the next layer in the network.
- Pros of using BN:
 - Reduce the number of epochs it takes to train a neural network.
 - “Stabilize” training, allowing for a larger variety of learning rates and regularization strengths.
 - You’ll also tend to notice lower final loss and a more stable loss curve when using batch normalization in your networks.
- Cons of using BN:
 - Slow down the wall time it takes to train your network.

Batch Normalization

- If we consider x to be our mini-batch of activations, then:

$$\hat{x}_i = \frac{x_i - \mu_\beta}{\sqrt{\sigma_\beta^2 + \epsilon}}$$

- During training phase:

$$\mu_\beta = \frac{1}{M} \sum_{i=1}^m x_i \qquad \sigma_\beta^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_\beta)^2$$

- During prediction phase:
 - Replace the mini-batch μ_β and σ_β with running averages of μ_β and σ_β computed during the training process.

Dropout

- Dropout is actually a form of regularization that aims to help prevent overfitting by increasing testing accuracy.
- For each mini-batch in our training set, **dropout layers, with probability p , randomly disconnect inputs from the preceding layer to the next layer in the network architecture.**
- It is most common to place dropout layers in-between FC layers.

